Evaluation: Gate Criteria, QA Probes, and Failure Discovery

Evaluation: Gate Criteria, QA Probes, and Failure Discovery I

The evaluation section is configured to name readiness criteria, connect criteria to artifacts, separate local checks from publication readiness, and make failure probes visible. The local readiness surface is not a human preference score; it is a set of deterministic checks that connect generated manuscript claims to source files and pipeline gates.

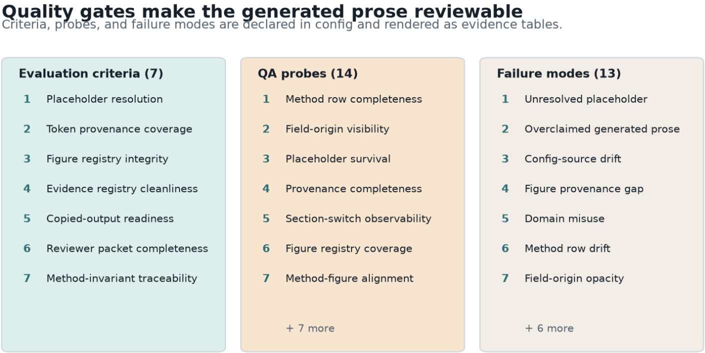
The active criteria use analysis, copy, pytest, validation as gate labels and inspect artifacts such as token-injection flow, quality-gate matrix, configured-field figures. A passing run means the exemplar is locally render-ready: placeholders resolve, token provenance is present, figure references are registered, evidence scanning has not found unsupported numbers, and project design overlays remain internally consistent.

Evaluation: Gate Criteria, QA Probes, and Failure Discovery I

That readiness is deliberately narrower than publication readiness. A local pass does not imply reader-preference results or empirical validation. It means the tracked project tree can regenerate the committed artifact surface through its declared pipeline.

The QA probes are Method row completeness, Field-origin visibility, Placeholder survival, Provenance completeness, Section-switch observability, Figure registry coverage, Method-figure alignment, Evidence cleanliness, Fork readiness, Copied-output parity, Digest invariant review, Claim-ledger alignment, Review packet completeness, Fork migration sufficiency. They are phrased as questions so they can be reused by reviewers and by forks of the exemplar: did the placeholder disappear, did the provenance survive, did the figure registry cover every reference, and did copied outputs preserve the same evidence surface that validation inspected?

Evaluation: Gate Criteria, QA Probes, and Failure Discovery I



Quality gates make the generated prose reviewable

Criteria, probes, and failure modes are declared in config and rendered as evidence tables.

Evaluation criteria (7)

1. Placeholder resolution
2. Token provenance coverage
3. Figure registry integrity
4. Evidence registry cleanliness
5. Copied-output readiness
6. Reviewer packet completeness
7. Method-invariant traceability

QA probes (14)

1. Method row completeness
2. Field-origin visibility
3. Placeholder survival
4. Provenance completeness
5. Section-switch observability
6. Figure registry coverage
7. Method-figure alignment

+ 7 more

Failure modes (13)

1. Unresolved placeholder
2. Overclaimed generated prose
3. Config-source drift
4. Figure provenance gap
5. Domain misuse
6. Method row drift
7. Field-origin opacity

+ 6 more

Evaluation: Gate Criteria, QA Probes, and Failure Discovery I

Figure 8: Quality gate matrix

Evaluation Criteria I

Criterion	Target	Evidence	Gate
Placeholder resolution	No unresolved uppercase manuscript placeholders remain in output/manuscript or rendered web output.	tests/test_manuscript_variables.py and rg unresolved-token scan.	`pytest`
Token provenance coverage	Every selected token maps to category, selected value, section, and config pointer.	output/reports/injection_trace.json and output/data/token_inventory.json.	`analysis`
Figure registry integrity	Every referenced figure label is present in output/figures/figure_registry.json.	scripts/pipeline/stage_04_validate.py figure registry check.	`validation`
Evidence registry cleanliness	Generated manuscript numbers and claims pass the project evidence registry.	output/reports/evidence_registry.json	`validation`
Copied-output readiness	PDF, HTML, slides, figures, data, and reports copy into output/templates/template_madlib.	scripts/pipeline/stage_05_copy.py output statistics.	`copy`
Reviewer packet completeness	Hydrated Markdown, rendered PDF, web output, slides, figures, data, reports, validation results, and copy statistics are all present for review.	Stage 04 validation report and Stage 05 output_statistics.json.	`copy`
Method-invariant traceability	Token choices can be explained only by seed, slot, category, ordinal, and ordered category inventory.	tests/test_tokens.py and generated Methods digest prose.	`pytest`

Quality Probes I

Probe	Question	Passing signal	Artifact
Method row completeness	Does the protocol table cover schema intake, token planning, composition, figures, validation, copy, and review handoff?	method_protocol includes rows for every major pipeline responsibility.	`manuscript/config.yaml and output/data/section_plan.json`
Field-origin visibility	Can a reviewer tell which visible fields were authored and which were defaulted?	Configured-field inventory and summary tables report explicit and defaulted paths.	`output/data/configured_field_inventory.json`
Placeholder survival	Did any source token survive hydration?	No uppercase placeholders are found in generated manuscript or web files.	`output/manuscript and output/web`
Provenance completeness	Can every selected token be traced to a category, section, value, and config key?	The injection trace and token inventory contain one row for each generated token.	`output/reports/injection_trace.json`
Section-switch observability	Does a disabled section resolve to a visible explanation?	Disabled section bodies cite their controlling section condition.	`output/data/section_plan.json`
Figure registry coverage	Does every manuscript figure reference have a generated registry entry?	Figure registry validation passes.	`output/figures/figure_registry.json`
Method-figure alignment	Do method figures describe generated data rather than decorative or unsupported claims?	Figure registry captions, nonblank PNG tests, and manual visual QA align with artifact data.	`output/figures`
Evidence cleanliness	Do generated claims stay within local evidence boundaries?	Evidence registry validation passes without unsupported claims.	`output/reports/evidence_registry.json`

Quality Probes I

Probe	Question	Passing signal	Artifact
Fork readiness	Does the authoring contract tell downstream forks what to extend before adding domain claims?	Authoring obligations cite config diffs, claim ledger updates, validators, and full reruns.	`output/manuscript/10_authoring_contract.md`
Copied-output parity	Did copied deliverables preserve the validated project output surface?	Copy-stage statistics include PDF, HTML, slides, figures, data, and reports.	`output/templates/template_madlib`
Digest invariant review	Are the allowed token-selection inputs documented and protected by tests?	Methods prose names the digest inputs and token tests prove seed/category sensitivity.	`src/template_madlib/tokens.py` and `output/manuscript/02_methodology.md`
Claim-ledger alignment	Do method and documentation claims point to config, source, generated artifacts, or explicit non-claim boundaries?	Claim-ledger rows cover expanded method protocol and fork-validator boundaries.	`data/claim_ledger.yaml`
Review packet completeness	Can a reviewer inspect every output surface needed to audit the method?	Copied outputs include manuscript, web, slides, figures, data, reports, validation, and copy statistics.	`output/templates/template_madlib` and `output/reports/output_statistics.json`
Fork migration sufficiency	Does the documentation tell forks which surfaces to change before adding domain claims?	README, STANDALONE, manuscript README, and Authoring Contract list config, source, test, validator, and claim-ledger obligations.	`README.md`, `STANDALONE.md`, `manuscript/README.md`, and `output/manuscript/10_authoring_contract.md`